

PROGRAMA DE SESIONES DE CLASE

Taller de Herramientas Computacionales

Propuesta de organización didáctica para estudiantes de primer semestre de licenciatura de Matemáticas Aplicadas.

Athziri Hernández Jiménez

Duración total:	64 horas
Número de sesiones:	32 sesiones
Duración por sesión:	2 horas
Modalidad:	Teórico-práctica
Herramientas principales:	Python, <i>Google Colab</i> , <i>Visual Studio Code</i> , <i>GitHub</i> , L ^A T _E X y <i>Overleaf</i>

Contents

1	Presentación del programa	2
2	Objetivo general	2
3	Objetivos específicos	2
4	Organización general del curso	3
5	Estructura sugerida de una sesión de dos horas	3
6	Programa de sesiones	3
6.1	Bloque I. Introducción a Python y pensamiento computacional	5
6.2	Bloque II. Aplicaciones y experimentación computacional	9
6.3	Bloque III. Introducción a Linux y GitHub	11
6.4	Bloque IV. Introducción a LaTeX y comunicación científica	13
7	Propuesta de ejercicios integradores	15
8	Proyecto final	15
8.1	Producto final esperado y entregables	16
9	Consideraciones didácticas finales	17
10	Recursos de consulta recomendados	17

1 Presentación del programa

El presente programa propone una organización progresiva de las sesiones del curso *Taller de Herramientas Computacionales*, dirigida a estudiantes de primer semestre de licenciatura en Matemáticas Aplicadas.

La propuesta parte del supuesto de que los estudiantes poseen niveles heterogéneos de experiencia previa con computadoras y programación. La secuencia inicia con actividades de familiarización con el entorno de trabajo y resolución de problemas sencillos, posteriormente avanzar hacia la programación estructurada, el manejo de datos, la representación gráfica, el cálculo simbólico, aplicaciones matemáticas y computacionales, a la vez, la elaboración de documentos científicos mediante $\text{\LaTeX}$.

Durante el curso se pretenden utilizar cuatro entornos principales:

- **Google Colab:** para introducir la programación sin requerir instalación local y facilitar el trabajo mediante cuadernos interactivos.
- **Visual Studio Code:** para introducir gradualmente el trabajo con archivos de código, scripts, organización de proyectos y ejecución local de programas.
- **GitHub:** para introducir principios básicos de control de versiones, documentación de proyectos y trabajo reproducible.
- **Overleaf:** para la elaboración colaborativa de documentos, reportes técnicos, fórmulas matemáticas, tablas, figuras y presentaciones.

2 Objetivo general

Introducir al estudiante en el uso informado de herramientas computacionales para resolver, analizar y comunicar problemas matemáticos y científicos, mediante el aprendizaje progresivo de programación en **Python**, el desarrollo de aplicaciones y experimentos computacionales, el uso básico de control de versiones con *GitHub* y la elaboración de documentos técnicos y científicos mediante $\text{\LaTeX}$.

3 Objetivos específicos

Al finalizar el curso, se espera que el estudiante sea capaz de:

1. Reconocer los elementos básicos de un lenguaje de programación.
2. Resolver problemas sencillos mediante algoritmos y programas escritos en **Python**.
3. Utilizar variables, tipos de datos, funciones, estructuras condicionales, ciclos, vectores y matrices.
4. Manipular y visualizar datos mediante herramientas computacionales.
5. Representar gráficamente funciones y resultados numéricos.
6. Utilizar herramientas básicas de cálculo simbólico.
7. Aplicar métodos computacionales a problemas matemáticos, geométricos y científicos.
8. Reconocer la importancia de los errores numéricos, el redondeo y la precisión computacional.

9. Generar y utilizar números aleatorios en simulaciones sencillas.
10. Utilizar *GitHub* como herramienta básica para almacenar, organizar y versionar proyectos.
11. Elaborar documentos académicos y científicos utilizando $\text{\LaTeX}$ en *Overleaf*.
12. Integrar programación y documentación en un proyecto final.

4 Organización general del curso

	Bloque	Temática principal
El curso se organiza en cuatro bloques principales.	I	Introducción a la programación y pensamiento
	II	Aplicaciones, visualización y experimentación
	III	Introducción a Linux y <i>GitHub</i>
	IV	Introducción a $\text{\LaTeX}$ y comunicación científica

5 Estructura sugerida de una sesión de dos horas

La estructura puede modificarse dependiendo de la complejidad del tema, pero se propone la siguiente organización general:

Actividad	Tiempo	Propósito
Inicio y recuperación de conocimientos	10 min	Relacionar el tema con conocimientos previos y presentar el problema.
Introducción conceptual	20–25 min	Explicar los conceptos fundamentales mediante ejemplos breves.
Demostración guiada	20–25 min	Mostrar el procedimiento de programación o uso de la herramienta.
Actividad práctica	35–45 min	Resolver ejercicios individuales o en parejas.
Discusión y revisión	10–15 min	Analizar resultados, errores y estrategias de solución.
Cierre	5 min	Recuperar los conceptos centrales y presentar la actividad siguiente.

6 Programa de sesiones

Lista de sesiones

- Sesión 1. Presentación del curso.
- Sesión 2. Introducción al pensamiento computacional y primer contacto con Python. Variables y tipos de datos.
- Sesión 3. Entrada y salida de información.
- Sesión 4. Funciones básicas y organización del código. Estructuras condicionales.

- Sesión 5. Ciclos y repetición controlada.
- Sesión 6. Listas y manejo básico de colecciones de datos.
- Sesión 7 Práctica integradora previa a la primera evaluación: resolución de problemas utilizando variables, condicionales, ciclos, listas y funciones.
- Sesión 8. **Primera Evaluación: Fundamentos de programación y fundamentos de LaTeX: primer texto.**
- Sesión 9. Arreglos numéricos con NumPy y operaciones vectorizadas.
- Sesión 10. Matrices y arreglos bidimensionales.
- Sesión 11. Recursiones
- Sesión 12. Cálculo simbólico con SymPy.
- Sesión 13. Resolución numérica y gráfica de problemas: raíces y ecuaciones.
- Sesión 14. Graficación de funciones. Graficación y visualización de datos y funciones con Matplotlib.
- Sesión 15. Errores numéricos, precisión y redondeo.
- Sesión 16. Números aleatorios y simulación.
- Sesión 17. Representación geométrica y transformaciones en 2D y 3D; Fractales y construcción del Triángulo de Sierpinski. **Temas comodín**
- Sesión 18. Práctica integradora previo a la segunda evaluación: modelación, análisis y visualización de un problema matemático o científico.
- Sesión 19: **Segunda evaluación: aplicación de herramientas computacionales para el análisis y resolución de problemas. LaTeX: modo matemático.**
- Sesión 20. Introducción a Linux y GitHub: repositorios, archivos y organización de proyectos. Control de versiones y organización de proyectos con GitHub.
- Sesión 1. Introducción a LaTeX y Overleaf. Estructura y formato de documentos en LaTeX.
- Sesión 2. Listas, tablas y organización de resultados.
- Sesión 3. Figuras, gráficas y referencias cruzadas.
- Sesión 4. Escritura de expresiones matemáticas en LaTeX.
- Sesión 5. Teoremas, Corolarios, demostraciones, etc.
- Sesión 6. Referencias bibliográficas y citas.
- Sesión 7. Presentación con Beamer.

- Sesión Final. Proyecto de aplicación computacional.

Calendario	Python	L ^A T _E X
Septiembre	8,22,29	3,10,17,24
Octubre	6,13,15,27	1,8, 29
1era. Evaluación:	20 y 22	
Noviembre	3, 5, 10, 12, 17, 19, 24 y 26	
Diciembre: 2da. Evaluación:	1 y 4	
Diciembre: Linux y GitHub	8 y 9	
Enero: Exposiciones proyecto final	4, 9, 11 y 16	

6.1 Bloque I. Introducción a Python y pensamiento computacional

Sesión 1. Presentación del curso y evaluación de conocimientos generales

Temática:

- Presentación de la asignatura.
- Realización de un cuestionario evaluando los conocimientos generales de los alumnos.

Objetivo de la sesión:

Conocer el estado de conocimiento sobre programación de los alumnos.

Sesión 2a. Introducción al pensamiento computacional y primer contacto con Python.

Temática:

- Presentación de la asignatura.
- ¿Qué es un algoritmo?
- ¿Qué es un lenguaje de programación?
- Primer contacto con Python mediante *Google Colab*.

Objetivo de la sesión:

Que el estudiante reconozca la relación entre un problema, un algoritmo y un programa, y ejecute sus primeras instrucciones en Python.

Ejercicios:

1. Escribir un algoritmo para preparar una bebida, sandwich, pasta, etc.
2. Identificar instrucciones ambiguas.
3. Ejecutar un programa que calcule operaciones aritméticas.

Sesión 2b. Variables y tipos de datos

Temática:

- Variables.
- Números enteros y reales.

- Cadenas de texto.
- Operadores aritméticos.
- Asignación.

Ejercicios:

1. Calcular el área de un rectángulo.
2. Calcular el perímetro de un círculo.
3. Convertir una temperatura de grados Celsius a Fahrenheit.
4. Calcular la distancia recorrida en un movimiento con velocidad constante.

Actividad práctica integradora:

Construir un programa que reciba valores definidos por el usuario y calcule una cantidad física o matemática sencilla.

Sesión 3. Entrada y salida de información

Temática:

- Función `input()`.
- Conversión de tipos.
- Salida de información.
- Formato de resultados.

Ejercicios:

1. Calculadora de índice de masa corporal.
2. Conversor de unidades.
3. Cálculo del promedio de tres calificaciones.
4. Cálculo de la ecuación de movimiento rectilíneo uniforme.

Sesión 4a. Funciones básicas y organización del código

Temática:

- Funciones matemáticas.
- Importación de módulos.
- Funciones definidas por el usuario.
- Parámetros y valores de retorno.

Ejercicios:

1. Crear una función para calcular el área de un triángulo.
2. Crear una función para convertir kilómetros a metros.
3. Crear una función para calcular la velocidad media.
4. Crear una función que determine si un número es positivo, negativo o cero.

Sesión 4b. Estructuras condicionales

Temática:

- Expresiones booleanas.
- Operadores relacionales.
- `if`.
- `if-else`.
- `if-elif-else`.

Ejercicios:

1. Determinar si un número es par o impar.
2. Determinar el mayor de dos números.
3. Clasificar una temperatura como baja, media o alta.
4. Resolver un problema de tarifa con diferentes condiciones.

Sesión 5. Ciclos: repetición controlada

Temática:

- Concepto de iteración.
- Ciclo `for`.
- Función `range()`.
- Ciclo `while`.
- Variables acumuladoras.
- Contadores.
- Condicionales dentro de ciclos.
- Diseño de algoritmos.

Ejercicios:

1. Mostrar los números del 1 al 100.
2. Calcular la suma de los primeros n números.
3. Generar una tabla de multiplicar.
4. Aproximar la suma de una serie matemática.
5. Desarrollar una versión simple de un programa de análisis de calificaciones.

Sesión 6. Listas y manejo básico de colecciones de datos

Temática:

- Listas.
- Índices.
- Acceso y modificación de elementos.
- Recorrido de listas.

Ejercicios:

1. Calcular el máximo y mínimo de una lista.
2. Calcular el promedio de una lista.
3. Contar cuántos valores cumplen una condición.
4. Construir una lista de cuadrados de números.

Sesión 7. Práctica Integradora

Sesión 8. Primera Evaluación

Sesión 9. Arreglos numéricos con NumPy y operaciones vectorizadas

Temática:

- Introducción a NumPy.
- Arreglos unidimensionales.
- Operaciones vectorizadas.
- Generación de secuencias numéricas.

Ejercicios:

Generar valores de posición para:

1. Movimiento rectilíneo uniforme.
2. Caída libre ideal.
3. Movimiento con aceleración constante.

Sesión 10. Matrices y arreglos bidimensionales

Temática:

- Matrices.
- Dimensiones.
- Índices.
- Operaciones básicas.
- Producto matricial.

Ejercicios:

1. Crear matrices de diferentes dimensiones.
2. Calcular la suma de dos matrices.
3. Calcular el producto de una matriz por un escalar.
4. Explorar una transformación lineal sencilla.

Sesión 11a. Funciones, modularidad y reutilización

Temática:

- Diseño de funciones.
- Separación de problemas.
- Reutilización de código.
- Pruebas básicas.

Ejercicios:

Desarrollar una pequeña biblioteca de funciones para:

- Conversión de unidades.
- Áreas y perímetros.
- Velocidad y aceleración media.
- Operaciones estadísticas sencillas.

Sesión 11b. Ceros de una función y raíces de polinomios

Temática:

- Interpretación gráfica de una raíz.
- Raíces de polinomios.
- Métodos numéricos sencillos.
- Comparación entre resultados simbólicos y numéricos.

Ejercicios:

1. Encontrar las raíces de un polinomio cuadrático.
2. Explorar polinomios de mayor grado.
3. Graficar la función e identificar sus ceros.
4. Comparar una solución gráfica, simbólica y numérica.

Sesión 11c. Recursión

Temática:

- Concepto de recursión.
- Caso base.
- Llamada recursiva.

Ejercicios:

1. Factorial.
2. Sucesión de Fibonacci.
3. Suma de los primeros n enteros.
4. Comparar una solución iterativa con una recursiva.

Sesión 12. Cálculo simbólico con SymPy

Temática:

- Introducción a SymPy.
- Variables simbólicas.
- Simplificación.
- Derivación.
- Integración.
- Resolución de ecuaciones sencillas.

Ejercicios:

Resolver simbólicamente y verificar numéricamente problemas elementales de cálculo.

6.2 Bloque II. Aplicaciones y experimentación computacional

Sesión 14. Graficación de funciones, Visualización de datos y funciones con Matplotlib

Temática:

- Introducción a Matplotlib.
- Gráficas de funciones.
- Ejes, etiquetas y leyendas.

Actividad práctica:

Representar gráficamente diferentes funciones y analizar cómo cambian sus gráficas al modificar parámetros.

Sesión 15. Errores numéricos, precisión y redondeo.

Temática:

- Precisión finita.
- Error absoluto.
- Error relativo.
- Propagación básica de errores.
- Problemas asociados al redondeo.

Ejercicios:

1. Comparar aproximaciones de π .
2. Analizar el error al redondear valores.
3. Comparar diferentes representaciones numéricas.
4. Analizar la diferencia entre dos expresiones matemáticamente equivalentes.

Sesión 16. Números aleatorios y simulación

Temática:

- Generación de números aleatorios.
- Distribuciones básicas.
- Simulación computacional.

Ejercicio:

Realizar una estimación de una cantidad mediante simulación de Monte Carlo sencilla.

Sesión 17a. Representación geométrica de transformaciones en 2D, Optativo

Temática:

- Vectores en el plano.
- Transformaciones lineales.
- Escalamiento.
- Rotación.
- Reflexión.

Ejercicio:

Representar una figura geométrica y aplicar una matriz de transformación para observar sus cambios.

Sesión 17b. Transformaciones en 3D. **Optativo**

Temática:

- Coordenadas tridimensionales.
- Representación de superficies.
- Transformaciones geométricas.

Ejercicios:

1. Graficar una superficie sencilla.
2. Representar un paraboloide.
3. Explorar una esfera.
4. Aplicar una transformación a un conjunto de puntos.

Sesión 17c. Fractales y el Triángulo de Sierpinski, **Optativo**

Temática:

- Autosimilitud.
- Algoritmos recursivos.
- Construcción computacional de fractales.

Actividad principal:

Construcción del Triángulo de Sierpinski mediante una estrategia iterativa o recursiva.

Ejercicio de ampliación:

Explorar otros patrones fractales y comparar el algoritmo utilizado.

Sesión 18: Práctica integradora

Objetivo:

Integrar los conocimientos de programación, funciones, ciclos, arreglos y visualización.

Posibles proyectos:

1. Simulación de caída libre.
2. Exploración de trayectorias de un proyectil.
3. Análisis de una función matemática con parámetros.
4. Simulación aleatoria.
5. Generación de un fractal.
6. Análisis básico de un conjunto de datos.

Producto de la sesión:

Un cuaderno de *Google Colab* organizado, comentado y funcional.

Sesión 19: Segunda Evaluación

6.3 Bloque III. Introducción a Linux y GitHub

Sesión 20. ¿Qué es Linux y qué es GitHub?

Temática:

- ¿Qué es Linux y qué es un sistema operativo?
- Distribuciones de Linux y relación con otros sistemas operativos.
- Estructura básica del sistema de archivos.
- Introducción a la terminal y la línea de comandos.
- Navegación entre directorios.
- Comandos básicos: `pwd`, `ls`, `cd`, `mkdir`, `cp` y `rm`.
- Problemas de trabajar con archivos como: `proyecto_final.py`, `proyecto_final_ahora_si.py`, `proyecto_final_ultimo.py`.
- Repositorios.
- Archivos y carpetas.
- README.

Actividad práctica Linux:

- Exploración del sistema: utilizar los comandos `pwd`, `ls` y `cd` para identificar la ubicación actual, listar los archivos y navegar entre diferentes directorios.
- Organización de archivos: crear mediante la terminal una carpeta llamada `cursoPython`, y dentro de ella las subcarpetas `ejercicios`, `proyectos` y `datos`.
- Gestión básica de archivos: crear un archivo de texto, copiarlo a otra carpeta, cambiarle el nombre y finalmente eliminar una copia utilizando comandos de la terminal.
- Mostrar una situación en la que existen varias versiones de un mismo archivo y discutir las dificultades para identificar cuál contiene la versión correcta.

Actividad práctica Github:

1. Crear una cuenta en *GitHub*.
2. Crear un primer repositorio.
3. Agregar un archivo `README.md`.
4. Describir brevemente un proyecto.
5. Subir un archivo de Python.

Ejercicio:

Cada estudiante crea un repositorio llamado, por ejemplo, `taller-herramientas-computacionales` y organiza una estructura inicial:

```
taller-herramientas-computacionales/
|
|-- README.md
|-- python/
|-- ejercicios/
|-- proyecto/
|-- documentos/
```

6.4 Bloque IV. Introducción a LaTeX y comunicación científica

Sesión 1. Introducción a LaTeX y Overleaf. Estructura de un documento.

Temática:

- ¿Qué es $\text{\LaTeX}$?
- Diferencia entre editor visual y sistema de composición.
- Uso inicial de *Overleaf*.
- Estructura básica de un documento.
- Secciones y subsecciones.
- Párrafos.
- Énfasis tipográfico.
- Paquetes.
- Comentarios.
- Compilación.

Ejercicio:

Modificar una plantilla sencilla que contenga:

- Título.
- Autor.
- Fecha.
- Secciones.
- Texto.

Sesión 2. Listas, tablas y organización de resultados

Temática:

- Listas numeradas.
- Listas con viñetas.
- Tablas.
- Alineación de datos.
- Uso de tablas para resultados.

Actividad práctica:

Construir una tabla de datos generados previamente en `Python` y presentarla correctamente en un documento.

Sesión 3. Figuras, gráficas y referencias cruzadas

Temática:

- Inclusión de figuras.
- Tamaño y posición.
- Leyendas.
- Etiquetas.
- Referencias cruzadas.

Actividad integradora:

Generar una gráfica en **Python**, guardarla como imagen e incluirla en un documento de $\text{\LaTeX}$.
El estudiante deberá explicar:

1. Qué representa la gráfica.
2. Qué variables se presentan.
3. Qué comportamiento se observa.

Sesión 4. Escritura de expresiones matemáticas**Temática:**

- Modo matemático.
- Superíndices y subíndices.
- Fracciones.
- Raíces.
- Ecuaciones.
- Símbolos matemáticos.

Ejercicios:

Escribir en $\text{\LaTeX}$:

$$v = \frac{\Delta x}{\Delta t} \quad (1)$$

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2 \quad (2)$$

$$\int_a^b f(x) dx \quad (3)$$

Sesión 5. Teoremas, Corolarios, demostraciones, etc.**Sesión 6. Referencias bibliográficas y citas****Temática:**

- Importancia de citar fuentes.
- Referencias bibliográficas.
- Archivos **.bib**.
- Citas dentro del texto.
- Introducción a BibTeX y Biber.

Actividad práctica:

Crear una pequeña base bibliográfica con tres referencias y citarlas dentro de un documento.

Sesión 7. Presentacion Beamer

Temática:

- Introducción a presentaciones con Beamer.
- Estructura de una presentación científica.
- Integración de código, resultados y documentación.

Actividad final:

Presentar un proyecto breve que integre:

1. Un problema matemático o científico.
2. Código desarrollado en **Python**.
3. Una o más representaciones gráficas.
4. Un repositorio organizado en *GitHub*.
5. Un reporte escrito en $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$.
6. Una presentación breve.

7 Propuesta de ejercicios integradores

Falta proponer

8 Proyecto final

El proyecto final consiste en realizar una breve investigación sobre un tema de interés relacionado con las matemáticas aplicadas a cualquier área de estudio (ciencias, ingeniería, economía, ciencias sociales, entre otras). El trabajo deberá incluir una propuesta de análisis, modelo o solución en la que se utilicen herramientas de programación —como **Python**— para el procesamiento, simulación o visualización de resultados. El proyecto deberá documentarse en un **informe elaborado en LaTeX** y exponerse ante el grupo mediante **Beamer**.

El proyecto deberá permitir identificar claramente:

- Qué problema se estudia.
- Qué modelo matemático se utiliza.
- Cómo se ejecuta el programa.
- Qué resultados se obtienen.
- Cómo se interpretan dichos resultados.
- Qué fuentes bibliográficas se utilizaron.

Como mínimo deberá contener:

1. Planteamiento del problema.
2. Objetivos del proyecto.
3. Fundamento teórico
4. Modelo matemático.
5. Código comentado.

6. Resultados numéricos o gráficos.
7. Análisis de resultados.
8. Conclusiones, opiniones finales o estado del arte.
9. Referencias y citas
10. Repositorio de *GitHub*.
11. Documento técnico elaborado en $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$.
12. Presentación breve en Beamer y oral.

8.1 Producto final esperado y entregables

Al finalizar el curso, cada estudiante deberá entregar su proyecto computacional organizado de manera similar a:

```
proyecto-final/
|
|-- README.md
|
|-- codigo/
|   |-- programa_principal.py
|   |-- funciones.py
|
|-- datos/
|
|-- figuras/
|
|-- documento/
|   |-- reporte.tex
|   |-- referencias.bib
|
|-- presentacion/
|   |-- presentacion.tex
```

Informe con la estructura:

- Introducción
- Fundamentos teóricos
- Metodología
- Resultados
- Conclusiones
- Bibliografía.
- Anexo: Código Python (**.ipynb** y **.pdf**) funcional y documentado.

El código es de cualquier cosa realizada con python expuesta en su proyecto: Gráficos, resolución de ecuaciones, etc.

Presentación Beamer (.pdf y .tex)

- Información sintetizada del informe técnico: Introducción, objetivos, resultados y su análisis, conclusiones y referencias.

- Recursos visuales: gráficos, fórmulas, etc.
- Extensión de entre 8 y 12 diapositivas.

Exposición oral

- Duración: máximo 10 minutos.
- Extensión: entre 8 y 12 diapositivas.

Evaluación: se considerará claridad, manejo del tiempo y conocimiento del tema.

9 Consideraciones didácticas finales

Dado que se trata de estudiantes de primer semestre, se recomienda evitar que el curso se convierta únicamente en una enseñanza de sintaxis. Cada concepto de programación debe asociarse con un problema que el estudiante pueda comprender, modificar y explorar.

Se propone la siguiente progresión pedagógica:

1. **Primero:** observar un ejemplo.
2. **Después:** modificarlo.
3. **Posteriormente:** reconstruirlo con ayuda.
4. **Finalmente:** resolver un problema nuevo de manera autónoma.

Asimismo, se recomienda mantener una conexión constante entre los diferentes entornos de trabajo:

Explorar → Programar → Organizar → Documentar → Comunicar

De esta manera, *Google Colab* puede utilizarse principalmente durante las primeras etapas de exploración y aprendizaje; *Visual Studio Code* para el desarrollo progresivo de programas y proyectos; *GitHub* para la organización y versionado del trabajo; y *Overleaf* para la elaboración de reportes y presentaciones.

10 Recursos de consulta recomendados

Para el desarrollo del curso se recomienda utilizar la documentación oficial y los recursos educativos correspondientes a:

- Python Software Foundation: documentación oficial de Python.
- Project Jupyter y Google Colaboratory: documentación sobre cuadernos computacionales.
- NumPy: documentación oficial para computación numérica.
- SymPy: documentación oficial para cálculo simbólico.
- Matplotlib: documentación oficial para visualización.
- Git: documentación y libro *Pro Git*.

- GitHub Docs: documentación sobre repositorios y control de versiones.
- Overleaf Learn: recursos para aprender $\text{\LaTeX}$.
- Comprehensive TeX Archive Network (CTAN): documentación y paquetes de $\text{\LaTeX}$.